

Binding Time	Description	Implications
Compile Time	If the memory location is known at compile time, absolute code is generated.	Requires recompilation if the starting location changes.
Load Time	If the memory location is known at load time but not at compile time, relocatable code is generated.	Final binding is delayed until load time. Only requires reloading the code if the starting address changes.
Execution Time	If the process can be moved from one memory segment to another during execution, binding is delayed until run time.	Requires special hardware support.

Logical vs. Physical Address Space

- **Logical Address:** An address generated by the CPU, also referred to as a virtual address.
- **Physical Address:** The address loaded into the memory address register of the memory unit.

The **logical address space** is the set of all addresses generated by a program, while the **physical address space** is the set of all physical addresses corresponding to those logical addresses.

In compile-time and load-time address-binding schemes, logical and physical addresses are the same. However, in the execution-time address-binding scheme, they differ.

Memory-Management Unit (MMU)

The run-time mapping from virtual to physical addresses is done by a hardware device called the MMU.

Many methods are possible for this mapping. A simple MMU scheme generalizes the base-register scheme, where the base register is called the **relocation register**. The value in the relocation register is added to every address generated by a user process at the time it is sent to memory.

For example, if the base is at 14000, an attempt to address location 0 is dynamically relocated to location 14000, and an access to location 346 is mapped to location 14346. The user program only sees logical addresses, while the memory mapping hardware converts these to physical addresses. Logical addresses range from 0 to max, and physical addresses range from $R+0$ to $R+max$, where R is the base value.

Dynamic Loading

Normally, the entire program and all data of a process must be in physical memory for the process to execute, limiting the size of a process to the size of physical memory. **Dynamic loading** is used to obtain better memory-space utilization.

With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in relocatable load format.

1. The main program is loaded into memory and executed.
2. When a routine needs to call another routine, the calling routine first checks if the other routine has been loaded.
3. If not, the relocatable linking loader is called to load the desired routine into memory and update the program's address tables.
4. Control is then passed to the newly loaded routine.

The advantage of dynamic loading is that unused routines are never loaded, which is useful when large amounts of code are needed for infrequently occurring cases (e.g., error routines). This is implemented through program design, and operating systems may provide library routines to support dynamic loading.

Dynamic Linking

- **Static Linking:** System libraries and program code are combined by the linker/loader into the binary program image.
- **Dynamic Linking:** Linking is postponed until execution time.

Dynamic linking is commonly used with system libraries. Without dynamic linking, each program must include a copy of its language library in the executable image, wasting both disk space and main memory.

A **stub** is included in the image for each library routine reference.

The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.

When the stub is executed:

1. It checks to see whether the needed routine is already in memory.
2. If not, the program loads the routine into memory.
3. The stub replaces itself with the address of the routine and executes the routine.
4. The next time that code segment is reached, the library routine is executed directly, without incurring the cost of dynamic linking.

Dynamic linking also facilitates library updates. A library can be replaced by a new version, and all programs that reference the library will automatically use the new version without needing to be relinked.

Swapping

Swapping is a process of temporarily swapping a process to a secondary memory from the main memory.

A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution. The **backing store** is a fast disk large enough to accommodate copies of all memory images for all users and must provide direct access to these memory images.

In a multiprogramming environment with a round-robin CPU-scheduling algorithm:

1. When a quantum expires, the memory manager swaps out the process that just finished and swaps another process into the freed memory space.
2. The CPU scheduler allocates a time slice to another process in memory.
3. This process repeats as each process finishes its quantum.

Ideally, the memory manager swaps processes fast enough so that processes are in memory, ready to execute, when the CPU scheduler wants to reschedule the CPU.

A variant of swapping used for priority-based scheduling algorithms is called **roll out, roll in**:

1. If a higher-priority process arrives and wants service, the memory manager swaps out the lower-priority process.
2. The higher-priority process is loaded and executed.
3. When the higher-priority process finishes, the lower-priority process is swapped back in and continued.

The major component of swap time is transfer time, which is directly proportional to the amount of memory swapped. The system maintains a ready queue of ready-to-run processes with memory images on disk.

Typically, a swapped-out process is swapped back into the same memory space it occupied previously. This restriction depends on the method of address binding. If binding is done at compile or load time, the process cannot be easily moved. If execution-time binding is used, a process can be swapped into a different memory space.

The dispatcher checks to see whether the next process in the queue is in memory. If it is not, the swapper swaps out a process currently in memory and swaps in the desired process. Then, it reloads registers and transfers control to the selected process.

The context-switch time in a swapping system can be significant. For example, swapping a 10 MB process to or from main memory using a hard disk with a transfer rate of 40 MB per second, with an average latency of 8 milliseconds, results in a swap time of 258 milliseconds. Considering both swap out and swap in, the total swap time is 516 milliseconds. For efficient CPU utilization, the execution time for each process should be long relative to the swap time.

Modified versions of swapping are found on many systems, such as UNIX, Linux, and Windows. Swapping is normally disabled and started only if memory allocation exceeds a threshold. It is disabled again once memory demand falls below the threshold.

Contiguous Allocation

Main memory must support both the operating system and user processes efficiently. Contiguous allocation, where each process is contained in a single contiguous section of memory, is one early method of memory allocation.

The memory is divided into two partitions:

- One for the resident operating system.
- One for the user processes.

The operating system can be located in either low or high memory. The location of the interrupt vector often dictates this decision, with the operating system commonly placed in low memory. User processes are then held in high memory.

To protect user processes from each other and from modifying operating-system code and data, a **relocation register** and a **limit register** are used.

1. The relocation register contains the value of the smallest physical address.
2. The limit register contains the range of logical addresses.

Each logical address must be less than the limit register, and the MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to memory.

Types of Memory Management Techniques

Memory management techniques can be broadly classified into two categories:

- **Contiguous Allocation:** The executing process must be loaded entirely in the main memory in a single, contiguous block.
 - **Fixed (or Static) Partitioning:** The number of partitions in RAM is fixed, although the size of each partition may vary.
 - **Variable (or Dynamic) Partitioning:** Partitions are created during run-time based on the needs of the processes.
- **Non-Contiguous Allocation:** A process is allocated a series of non-contiguous blocks of memory that can be located anywhere in the physical memory.

Fixed Partitioning

Also known as static partitioning, the number of partitions are fixed.

Fixed partitioning involves dividing the physical memory into fixed-size partitions or regions before execution or during system configuration. Each partition is dedicated to a specific process until it terminates or releases the partition.

A disadvantage of fixed partitioning is that it can lead to **internal fragmentation**, where memory within a partition remains unused because the process's memory requirements are smaller than the partition size.

Variable (or Dynamic) Partitioning

In contrast to fixed partitioning, partitions are not created before execution or during system configuration. Instead, partitions are made during run-time according to the process's needs.

Initially, RAM is empty, and partitions are created as processes arrive. The size of the partition is equal to the incoming process's needs, which avoids internal fragmentation and ensures efficient utilization of RAM. The number of partitions in RAM is not fixed and depends on the number of incoming processes and the size of the main memory.

Non-Contiguous Memory Allocation

In this technique, each process is allocated a series of non-contiguous blocks of memory that can be located anywhere in the physical memory. **Paging** and **segmentation** are two methods that allow a process's physical address space to be non-contiguous.

Non-contiguous memory allocation reduces memory wastage but increases overhead due to address translation, slowing down execution.

Memory Allocation - Fixed Size Partitions

One of the simplest methods for allocating memory is to divide it into several fixed-sized partitions, each of which can contain exactly one process. The degree of multiprogramming is limited by the number of partitions. When a partition is free, a process is selected from the input queue and loaded into the free partition. When the process terminates, the partition becomes available for another process. This method was originally used by the IBM OS/360 operating system (called MFT) but is no longer in use.

Memory Allocation - Variable Partition Scheme

In the variable partition scheme, the operating system maintains a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory—a **hole**. Over time, memory contains a set of holes of various sizes.

Hole: Block of available memory; holes of various size are scattered throughout memory

As processes enter the system, they are placed into an input queue. The operating system can order the input queue according to a scheduling algorithm. A process selected from the input queue is allocated memory from a hole large enough to accommodate it. The operating system can wait until a large enough block is available or skip down the input queue to see whether the smaller memory requirements of some other process can be met.

When a process terminates, it releases its block of memory, which is then placed back in the set of holes. If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. At this point, the system may need to check whether there are processes waiting for memory and whether this newly freed and re combined memory could satisfy the demands of any of these waiting processes.

Dynamic Storage-Allocation Problem

The dynamic storage-allocation problem involves satisfying a request of size **n** from a list of free holes. Several strategies exist:

- **First-Fit**: Allocate the first hole that is big enough.
- **Best-Fit**: Allocate the smallest hole that is big enough.
- **Worst-Fit**: Allocate the largest hole.

Algorithm	Description	Pros	Cons
First-Fit	Allocate the first hole that is big enough.	Simple and fast.	May result in larger holes being allocated, leading to fragmentation.
Best-Fit	Allocate the smallest hole that is big enough.	Minimizes the size of leftover holes.	Requires searching the entire list of holes, which can be slow.
Worst-Fit	Allocate the largest hole.	May leave larger holes for future allocations.	Requires searching the entire list and may lead to smaller, less useful holes over time.

Simulations have shown that both first-fit and best-fit perform better than worst-fit in terms of decreasing time and storage utilization. First-fit is generally faster than best-fit, but neither is clearly better in terms of storage utilization.

Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into smaller pieces, leading to fragmentation. There are two main types of fragmentation:

- External Fragmentation
- Internal Fragmentation

External Fragmentation

Total memory space exists to satisfy a request, but it is not contiguous.

Storage is fragmented into a large number of small holes. In the worst case, there is a block of free (or wasted) memory between every two processes.

Internal Fragmentation

Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition but not being used.

This is unused memory internal to a partition.

For example, consider a multiple-partition allocation scheme with a hole of 18,464 bytes. If the next process requests 18,462 bytes and we allocate exactly the requested block, we are left with a hole of 2 bytes, which may be unusable.

First-fit analysis reveals that given N blocks allocated, approximately $0.5N$ blocks are lost to fragmentation, with about $1/3$ being unusable, following the 50-percent rule.

Compaction

External fragmentation can be reduced by **compaction**, which involves shuffling memory contents to place all free memory together in one large block. Compaction is possible only if relocation is dynamic and done at execution time.

If addresses are relocated dynamically, relocation requires only moving the program and data and then changing the base register to reflect the new base address. The simplest compaction algorithm moves all processes toward one end of memory, with all holes moving in the other direction, producing one large hole of available memory. This scheme can be expensive.

Non-Contiguous Memory Allocation

Another solution to fragmentation is to permit the logical address space of processes to be non-contiguous, allowing a process to be allocated physical memory wherever such memory is available. Paging and segmentation are two ways to achieve this, reducing memory wastage but increasing overhead due to address translation.

Paging

Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous. This means that a process is allocated physical memory whenever it is available, avoiding external fragmentation and the need for compaction. It also avoids the problem of varying sized memory chunks.

Memory Division

Paging involves dividing both physical and logical memory: